

JavaScript Objects (Part 4)

This section covers:

- Looping Objects (`for...in`)
- Object Destructuring
- Spread Operator with Objects
- Real Project Examples
- Common Mistakes
- Interview Questions
- Summary Table

Looping Objects (for...in)

Definition

The **for...in** loop is used to iterate over the **keys (properties)** of an object.

It executes once for each enumerable property.

Syntax

```
for (const key in object) {  
  
    // code  
  
}
```

Example

```
const student = {  
  
    name: "Rokon",  
  
    age: 23,  
  
    cgpa: 3.90  
  
};  
  
for(const key in student){  
  
    console.log(key);  
  
}
```

OUTPUT

name

age

cgpa

Access Values

```
const student = {  
  
  name: "Rokon",  
  
  age: 23,  
  
  cgpa: 3.90  
  
};  
  
for(const key in student){  
  
  console.log(student[key]);  
  
}
```

OUTPUT

Rokon

23

3.9

Access Keys and Values

```
const student = {  
  
  name: "Rokon",  
  
  age: 23,  
  
  cgpa: 3.90  
  
};  
  
for(const key in student){  
  
  console.log(key, student[key]);  
  
}
```

OUTPUT

```
name Rokon
```

```
age 23
```

```
cgpa 3.9
```

Why Use Bracket Notation?

Inside a `for...in` loop, `key` is a variable.

```
student[key]
```

Correct

```
student.key
```

Wrong

Object Destructuring

Definition

Object destructuring extracts properties from an object and stores them in variables.

It makes code shorter and easier to read.

Without Destructuring

```
const user = {  
  name: "Rokon",  
  age: 23  
};  
  
const name = user.name;  
  
const age = user.age;  
  
console.log(name);  
  
console.log(age);
```

Output

Rokon

23

| With Destructuring

```
const user = {  
  name: "Rokon",  
  age: 23  
};  
  
const {name, age} = user;  
  
console.log(name);  
  
console.log(age);
```

OUTPUT

Rokon

23

Rename Variables

```
const user = {  
  
  name: "Rokon",  
  
  age: 23  
  
};  
  
const {  
  
  name: fullName,  
  
  age: userAge  
  
} = user;  
  
console.log(fullName);  
  
console.log(userAge);
```

OUTPUT

Rokon

23

Default Values

```
const user = {  
  name: "Rokon"  
};  
  
const {  
  name,  
  country = "Bangladesh"  
} = user;  
  
console.log(country);
```

OUTPUT

```
Bangladesh
```

Nested Object Destructuring

```
const student = {  
  
  name: "Rokon",  
  
  address: {  
  
    city: "Dhaka"  
  
  }  
  
};  
  
const {  
  
  address: {city}  
  
} = student;  
  
console.log(city);
```

OUTPUT

Dhaka

Spread Operator with Objects

Definition

The spread operator (`...`) copies or merges object properties into a new object.

Copy an Object

```
const user = {  
  name: "Rokon",  
  age: 23  
};  
  
const copy = {  
  ...user  
};  
  
console.log(copy);
```

OUTPUT

```
{  
  name: "Rokon",  
  age: 23  
}
```

Add New Property

```
const user = {  
  name: "Rokon",  
  age: 23  
};  
  
const updatedUser = {  
  ...user,  
  country: "Bangladesh"  
};  
  
console.log(updatedUser);
```

OUTPUT

```
{  
  name: "Rokon",  
  age: 23,  
  country: "Bangladesh"  
}
```

Update Existing Property

```
const user = {  
  
  name: "Rokon",  
  
  age: 23  
  
};  
  
const updatedUser = {  
  
  ...user,  
  
  age: 24  
  
};  
  
console.log(updatedUser);
```

OUTPUT

```
{  
  name: "Rokon",  
  age: 24  
}
```

Merge Objects

```
const personal = {  
  name: "Rokon"  
};  
  
const address = {  
  city: "Dhaka",  
  country: "Bangladesh"  
};  
  
const profile = {  
  ...personal,  
  ...address  
};  
  
console.log(profile);
```

OUTPUT

```
{  
  name: "Rokon",  
  city: "Dhaka",  
  country: "Bangladesh"  
}
```

Property Override

```
const user = {  
  
  name: "Rokon",  
  
  age: 23  
  
};  
  
const updated = {  
  
  ...user,  
  
  age: 25  
  
};  
  
console.log(updated);
```

OUTPUT

25

The last property with the same key overrides the previous one.

Real Project Example 1

| User Profile

```
const user = {  
  
  name: "Rokon",  
  
  email: "rokon@gmail.com",  
  
  role: "Admin"  
  
};  
  
for(const key in user){  
  
  console.log(`${key}: ${user[key]}`);  
  
}
```

Output

```
name: Rokon
```

```
email: rokon@gmail.com
```

```
role: Admin
```

Real Project Example 2

| Update User Information

```
const user = {  
  
  name: "Rokon",  
  
  age: 23  
  
};  
  
const updatedUser = {  
  
  ...user,  
  
  age: 24,  
  
  city: "Dhaka"  
  
};  
  
console.log(updatedUser);
```

Output

```
{  
  name: "Rokon",  
  age: 24,  
  city: "Dhaka"  
}
```

Real Project Example 3

Extract Product Data

```
const product = {  
  
  id: 1,  
  
  name: "Laptop",  
  
  price: 70000  
  
};  
  
const {  
  
  name,  
  
  price  
  
} = product;  
  
console.log(name);  
  
console.log(price);
```

Output

```
Laptop  
  
70000
```

Real Project Example 4

| Merge API Data

```
const user = {  
  
  id: 1,  
  
  name: "John"  
};  
  
const profile = {  
  
  country: "Bangladesh",  
  
  verified: true  
};  
  
const finalUser = {  
  
  ...user,  
  
  ...profile  
};  
  
console.log(finalUser);
```

Output

```
{  
  id: 1,  
  name: "John",  
  country: "Bangladesh",  
  verified: true  
}
```

Best Practices

✓ Use `for...in` only for objects.

✓ Use object destructuring to reduce repetitive code.

✓ Use the spread operator instead of manually copying properties.

✓ Prefer immutable updates using the spread operator in React.

```
const updatedUser = {  
  
  ...user,  
  
  age: 24  
  
};
```

Common Mistakes

| Using Dot Notation in for...in

Wrong

```
console.log(user.key);
```

Correct

```
console.log(user[key]);
```

| Forgetting Curly Braces in Destructuring

Wrong

```
const name = user;
```

Correct

```
const {name} = user;
```

Modifying the Original Object

Wrong

```
const copy = user;

copy.age = 30;
```

Both variables reference the same object.

Correct

```
const copy = {

  ...user

};
```

Overwriting Properties Accidentally

```
const result = {

  ...user,

  age: 25

};
```

The last value wins.

Interview Questions

| What does `for...in` loop iterate over?

Object keys (property names).

| Why use `user[key]` instead of `user.key` inside a `for...in` loop?

Because `key` is a variable containing the property name.

| What is Object Destructuring?

A syntax that extracts object properties into variables.

| What is the Spread Operator?

The spread operator (`...`) copies or merges objects.

Can the spread operator copy nested objects completely?

No.

It creates a **shallow copy**. Nested objects are still shared by reference.

Which property wins when merging objects with duplicate keys?

The **last** property overrides previous values.

```
const result = {  
  ...obj1,  
  ...obj2  
};
```

When is object destructuring commonly used?

- React Props
- API Responses
- Function Parameters
- Configuration Objects

Where is the spread operator commonly used?

- React State Updates
 - Redux
 - API Data Merging
 - Object Copying
-

Summary Table

Topic	Description	Example
<code>for...in</code>	Loop through object keys	<code>for (const key in user)</code>
Access Value	Get value using key	<code>user[key]</code>
Object Destructuring	Extract properties	<code>const {name} = user</code>
Rename Variable	Rename during destructuring	<code>{name: fullName}</code>
Default Value	Provide fallback	<code>{country = "BD"}</code>
Spread Operator	Copy object	<code>{...user}</code>
Merge Objects	Combine objects	<code>{...obj1, ...obj2}</code>
Override Property	Replace value	<code>{...user, age: 24}</code>

Most Frequently Used



- `for...in`

- Object Destructuring
- Spread Operator (`...`)



- Default Values
- Renaming Variables
- Merging Objects

Final Notes

These concepts are used daily in modern JavaScript, React, Node.js, and Express applications:

-  Loop through API response objects
-  Extract data from props and API responses
-  Update React state immutably
-  Merge configuration objects
-  Copy objects without modifying the original
-  Build scalable, maintainable applications

Mastering `for...in`, object destructuring, and the spread operator is essential for writing clean, modern JavaScript.